

ASR Confidential

ASR560X Series

Peripheral User's Guide

Version 1.2.0

Issue Date 2024-04-29

Copyright © 2024 ASR

About This Document

This document introduces the usage of all the peripherals of ASR560X.

Intended Readers

This document is mainly for engineers who use this chip to develop their own platform and products, for instance:

- PCB Hardware Development Engineer
- Software Engineer
- Technical Support Engineer

Included Chip Models

The product models corresponding to this document are as follows.

Model	Protocol	Core	SiP Flash	Function
ASR560X	BLE 5.1 full feature (compatible with 5.2) BLE SIG Mesh IEEE 802.15.4 2.4G Proprietary	ARM CM0+	1 MB/ 512 KB	AOA/AOD/Voice/IRTxRx/ Quadrature Decoder/Keypad/ 5V UART/5V GPIO/ Wi-Fi concurrent

ASR Confidential

Copyright Notice

© 2024 ASR Microelectronics Co., Ltd. All rights reserved. No part of this document can be reproduced, transmitted, transcribed, stored, or translated into any language in any form or by any means without the written permission of ASR Microelectronics Co., Ltd.

Trademark Statement

ASR and ASR Microelectronics Co., Ltd. are trademarks of ASR Microelectronics Co., Ltd.

Other trade names, trademarks, and registered trademarks mentioned in this document are the property of their respective owners and are hereby declared.

Electrostatic Discharge (ESD) Warning

This product can be damaged by Electrostatic Discharge (ESD). When handling with this device, the people should be very careful to conduct the ESD protection to avoid any device damage caused by ESD event.

Disclaimer

ASR does not give any warranty of any kind and may make improvements and/or changes in this document or in the product described in this document at any time.

This document is only used as a guide, and no contents in the document constitute any form of warranty. Information in this document is subject to change without notice.

All liability, including liability for infringement of any proprietary rights caused by using the information in this document, is disclaimed.

ASR Confidential

ASR Microelectronics Co., Ltd.

Address: 9F, Building 10, No. 399 Keyuan Road, Zhangjiang High-tech Park, Pudong New Area, Shanghai, 201203, China

Homepage: http://www.asrmicro.com/

Revision History

Date	Version	Release Notes
2020.06	V1.0.0	First Release.
2023.05	V1.1.0	Updated Chapter 4.
2024.04	V1.2.0	Updated some chapters.

Table of Contents

1. GPIO 1

1.1 Functional description 1

1.2 APIs 2

1.2.1 sonata_gpio_init 2

1.2.2 sonata_gpio_output_high 2

1.2.3 sonata_gpio_output_low 2

1.2.4 sonata_gpio_output_toggle 2

1.2.5 sonata_gpio_input_get 2

1.2.6 sonata_gpio_enable_irq 3

1.2.7 sonata_gpio_disable_irq 3

1.2.8 sonata_gpio_clear_irq 3

1.2.9 sonata_gpio_finalize 3

2. Watchdog 4

2.1 Functional description 4

2.2 APIs 4

2.2.1 sonata_wdg_init 4

2.2.2 sonata_wdg_reload 4

2.2.3 sonata_wdg_finalize 4

3. Timer 5

3.1 Functional description 5

3.2 APIs 6

3.2.1 sonata_timer_init 6

3.2.2 sonata_timer_start 6

3.2.3 sonata_timer_stop 6

3.2.4 sonata _timer_finalize 6

3.2.5 sonata_timer_get 6

3.2.6 sonata_timer_reload 7

3.2.7 sonata_inter_clear 7

3.2.8 sonata_get_curvalue 7

4. RTC 8

4.1 Functional description 8

4.2 APIs 9

4.2.1 sonata_rtc_init 9

4.2.2 sonata_rtc_get_time 9

4.2.3 sonata_rtc_set_time 9

4.2.4 sonata_rtc_finalize 9

5. ADC 10

5.1 Functional description 10

ASR Confidential

5.2 APIs 12
5.2.1 sonata_adc_init 12
5.2.2 sonata_aux_cmd 12
5.2.3 sonata_aux_int 12
5.2.4 sonata_get_aux_int_sta 12
5.2.5 sonata_clr_aux_int_sta 12
5.2.6 sonata_get_aux_fifo_sta 12
5.2.7 sonata_aud_cmd 13
5.2.8 sonata_aud_int 13
5.2.9 sonata_get_aud_int_sta 13
5.2.10 sonata_clr_aud_int_sta 13
5.2.11 sonata_get_aud_fifo_sta 13
5.2.12 sonata_aux_dma_ctl 13
5.2.13 sonata_aud_dma_ctl 13
5.2.14 sonata_aud_init_step 14
5.2.15 sonata_dma_adc_init 14
5.2.16 sonata_adc_init_SOFTFLOW 14
5.2.17 sonata_adc_finalize 14
5.2.18 sonata_get_aud_fifo_dat 14
5.2.19 sonata_get_aux_fifo_dat 14
6. PWM 15
6.1 Functional description 15
6.2 APIs 16
6.2.1 sonata_pwm_init 16
6.2.2 sonata_pwm_start 16
6.2.3 sonata_pwm_stop 16
6.2.4 sonata_pwm_finalize 16
6.2.5 sonata_cap_init 16
6.2.6 sonata_cap_finalize 17
6.2.7 sonata_pwm_para_chg 17
6.2.8 sonata_pwmdiv_config 17
6.2.9 sonata_pwm_clk_use_135M 17
7. Flash 18
7.1 Functional description 18
7.2 APIs 18
7.2.1 sonata_flash_init 18
7.2.2 sonata_flash_get_info 18
7.2.3 sonata_flash_find 19
7.2.4 sonata_flash_erase 19
7.2.5 sonata_flash_write 19
7.2.6 sonata_flash_erase_write 19
7.2.7 sonata_flash_read 20
7.3 Demo 20

ASR Confidential

8. Flash Encrypt 21
8.1 Functional description 21
8.2 APIs 21
8.2.1 sonata_flash_encrypt_status 21
8.2.2 sonata_flash_encrypt 21
8.3 Demo 21
9. Flash Remappig 22
9.1 Functional description 22
9.2 APIs 22
9.2.1 sonata_flash_remapping 22
9.3 Demo 22
10. UART 23
10.1 Functional description 23
10.2 APIs 24
10.2.1 sonata_uart_init 24
10.2.2 sonata_uart_set_tx_fifo_threshold 24
10.2.3 sonata_uart_set_rx_fifo_threshold 24
10.2.4 sonata_uart_interrupt_config 24
10.2.5 sonata_uart_cmd 25
10.2.6 sonata_uart_send 25
10.2.7 sonata_uart_dma_config 25
10.2.8 sonata_uart_start 25
10.2.9 sonata_uart_stop 26
10.2.10 sonata_uart_finalize 26
11. I2C 27
11.1 Functional description 27
11.2 APIs 28
11.2.1 sonata_i2c_unit_reset 28
11.2.2 sonata_i2c_config_struct_init 28
11.2.3 sonata_i2c_init 28
11.2.4 sonata_i2c_master_write_data 28
11.2.5 sonata_i2c_master_read_data 29
11.2.6 sonata_i2c_master_repeated_write_read 29
12. SPI 30
12.1 Functional description 30
12.2 APIs 30
12.2.1 sonata_spi_init 30
12.2.2 sonata_spi_send 30
12.2.3 sonata_spi_receive 31
12.2.4 sonata_spi_interrupt_config 31

ASR Confidential

12.2.5 sonata_spi_cmd 31
12.2.6 sonata_spi_finalize 31
12.2.7 sonata_spi_cpol_cpha_config 32
12.2.8 sonata_spi_dma_config 32
12.2.9 sonata_spi_datasize 32
12.2.10 sonata_spi_trx_byte 32
13. eFuse 33
13.1 Functional description 33
13.2 APIs 33
13.2.1 sonata_efuse_init 33
13.2.2 sonata_efuse_byte_read 33
13.2.3 sonata_efuse_word_read 33
13.2.4 sonata_efuse_byte_write 33
13.2.5 sonata_efuse_word_write 34
13.2.6 sonata_efuse_multi_read 34
13.3 Demo 34
14. AES 35
14.1 Functional description 35
14.2 APIs 35
14.2.1 sonata_aes_encrypt 35
14.2.2 sonata_aes_decrypt 35
14.3 Demo 36
15. SRAM Mapping 37
15.1 Functional description 37
15.2 Demo 37
16. I2S 38
16.1 Functional description 38
16.2 APIs 39
16.2.1 sonata_i2s_init 39
16.2.2 sonata_i2s_struct_init 39
16.2.3 sonata_i2s_send_data 39
16.2.4 sonata_i2s_interrupt_config 39
16.2.5 sonata_i2s_interrupt_clear 40
16.2.6 sonata_i2s_cmd 40
16.2.7 sonata_i2s_tx_block_cmd 40
16.2.8 sonata_i2s_rx_block_cmd 40
16.2.9 sonata_i2s_tx_channel_cmd 41
16.2.10 sonata_i2s_rx_channel_cmd 41
16.2.11 sonata_i2s_master_clock_cmd 41
16.2.12 sonata_i2s_receive_data 41
17. Keypad 42

ASR Confidential

17.1 Functional description 42
17.2 APIs 42
17.2.1 sonata_keypad_init 42
17.2.2 sonata_keypad_get_single_key_row 42
17.2.3 sonata_keypad_get_single_key_col 42
17.2.4 sonata_keypad_get_multi_key_stat 43
17.2.5 sonata_keypad_interrupt_config 43
17.2.6 sonata_keypad_cmd 43
17.2.7 sonata_keypad_get_status 43
17.2.8 sonata_keypad_wkup 43
17.2.9 sonata_keypad_get_multi_key_stat 43
17.2.10 sonata_keypad_scan 44
17.2.11 sonata_keypad_wkup_scan 44
18. LPUART 45
18.1 Functional description 45
18.2 APIs 46
18.2.1 sonata_lpuart_init 46
18.2.2 sonata_lpuart_config_rx 46
18.2.3 sonata_lpuart_config_tx 46
18.2.4 sonata_lpuart_config_interrupt 46
18.2.5 sonata_lpuart_receive_data 47
18.2.6 sonata_lpuart_send_data 47
18.2.7 sonata_lpuart_get_sr0_state 47
18.2.8 sonata_lpuart_clear_sr0_state 47
18.2.9 sonata_lpuart_get_sr1_state 48
18.2.10 sonata_lpuart_clear_sr1_state 48
18.2.11 sonata_lpuart_rxpin_sel 48
19. Quadrature Decoder 49
19.1 Functional description 49
19.2 APIs 49
19.2.1 sonata_quaddec_init 49
19.2.2 sonata_quaddec_get_dir 50
19.2.3 sonata_quaddec_get_cnt 50
19.2.4 sonata_quaddec_finalize 50
19.2.5 sonata_quaddec_set_seq 50
19.2.6 sonata_quaddec_get_seq 51

ASR Confidential

List of Tables

Table 1-1 sonata_gpio_dev_t structure 1
Table 2-1 sonata_wdg_dev_t structure 4
Table 3-1 sonata_timer_dev_t structure 5
Table 4-1 sonata_rtc_dev_t structure 8
Table 5-1 sonata_adc_dev_t structure 10
Table 6-1 sonata_pwm_dev_t structure 15
Table 6-2 sonata_cap_dev_t structure 15
Table 7-1 sonata_logic_partition_t structure 18
Table 10-1 sonata_uart_dev_t structure 23
Table 11-1 sonata_i2c_dev_t structure 27
Table 12-1sonata_spi_dev_t structure 30
Table 16-1 sonata_i2s_dev_t structure 38
Table 17-1 sonata_keypad_dev_t structure 42
Table 18-1 sonata_lpuart_dev_t structure 45
Table 19-1 sonata_quaddec_dev_t structure 49

ASR Confidential

1. GPIO

1.1 Functional description

ASR560X supports a total of 30 GPIOs for QFN48, and 14 GPIOs for QFN32, with the functions of input, output, pull control and interrupt. Structure **sonata_gpio_dev_t** is used to describe ASR560X GPIO hardware, and the definition is shown below:

Table 1-1 sonata_gpio_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	GPIO index, between 0 and 29
config	sonata_gpio_config_t	GPIO mode configuration: ● SONATA_ANALOG_MODE ● SONATA_IRQ_MODE ● SONATA_OUTPUT_PUSH_PULL ● SONATA_OUTPUT_OPEN_DRAIN_PULL_UP ● SONATA_OUTPUT_OPEN_DRAIN_NO_PULL ● SONATA_OUTPUT_PUSH_PULL_HIGH_CURRENT ● SONATA_INPUT_PULL_DOWN ● SONATA_INPUT_PULL_UP ● SONATA_INPUT_HIGH_IMPEDANCE
priv	void *	User-defined private data

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

1.2 APIs

1.2.1 sonata_gpio_init

Items	Description
Function	Initialize a GPIO pin, Prepare a GPIO pin for use
Parameters	● gpio: the GPIO device which should be initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.2 sonata_gpio_output_high

Items	Description
Function	Set an output GPIO pin high
Parameters	● gpio: the GPIO device which should be set high
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.3 sonata_gpio_output_low

Items	Description
Function	Set an output GPIO pin low
Parameters	● gpio: the GPIO device which should be set low
Return	Result: ● 0: success ● EIO : if an error occurs at any step

1.2.4 sonata_gpio_output_toggle

Items	Description
Function	Toggle the voltage of an output GPIO pin
Parameters	● gpio: the GPIO device which should be toggled
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.5 sonata_gpio_input_get

Items	Description
Function	Get the state of an input GPIO pin
Parameters	● gpio: the GPIO device which should be read ● value: used to store GPIO state

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

Items	Description
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.6 sonata_gpio_enable_irq

Items	Description
Function	Enable an interrupt trigger for an input GPIO pin
Parameters	● gpio: the GPIO device which will provide the interrupt trigger ● trigger: the type of trigger (rising/falling edge or high/low level) ● handler: a function pointer to the interrupt handler ● arg: an argument that will be passed to the interrupt handler
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.7 sonata_gpio_disable_irq

Items	Description
Function	Disable an interrupt trigger for an input GPIO pin
Parameters	● gpio: the GPIO device which will disable the interrupt trigger
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.8 sonata_gpio_clear_irq

Items	Description
Function	Clear interrupt flag for an input GPIO pin
Parameters	● gpio: the GPIO device which should be cleared interrupt flag
Return	Result: ● 0: success ● EIO: if an error occurs at any step

1.2.9 sonata_gpio_finalize

Items	Description
Function	Set a GPIO pin in default state
Parameters	● gpio: the GPIO device which should be de-initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

2. Watchdog

2.1 Functional description

The watchdog module applies a reset to a system in the event of a software failure, providing a way to recover from software crashes. Structure **sonata_wdg_dev_t** is used to describe ASR560X watchdog hardware, and the definition is shown below:

Table 2-1 sonata_wdg_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	watchdog index, should always be 0
config	sonata_wdg_config_t	watchdog configuration: ● timeout: watchdog timeout configuration
priv	void *	User-defined private data

2.2 APIs

2.2.1 sonata_wdg_init

Items	Description
Function	Initialize the watchdog
Parameters	● wdg: the watchdog device which should be initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

2.2.2 sonata_wdg_reload

Items	Description
Function	Reload watchdog counter
Parameters	● wdg: the watchdog device
Return	N/A

2.2.3 sonata_wdg_finalize

Items	Description
Function	De-initialize the watchdog
Parameters	● wdg: the watchdog device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

3. Timer

3.1 Functional description

ASR560X supports 4 timers whose function and operation are identical. For each timer, the following modes of operation are available:

- Periodic timer mode: The counter generates an interrupt at a constant interval, reloading the original value after wrapping past zero.
- One-shot timer mode: The counter generates an interrupt once. When the counter reaches 0, it halts until you reprogram it.

Structure **sonata_timer_dev_t** is used to describe ASR560X timer hardware, and the definition is shown below:

Table 3-1 sonata_timer_dev_t structure

Member Variable Name	Type	Description
port	int8_t	timer index, should be 0 to 3
config	sonata_timer_config_t	timer configuration: ● period: unit in us ● reload_mode: TIMER_RELOAD_AUTO: Periodic timer mode TIMER_RELOAD_MANU: One-shot timer mode ● cb: callback function when the timer times out ● arg: arguments passed to callback function
priv	void *	User-defined private data

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

3.2 APIs

3.2.1 sonata_timer_init

Items	Description
Function	Initialize a hardware timer
Parameters	● tim: the timer device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

3.2.2 sonata_timer_start

Items	Description
Function	Start a hardware timer
Parameters	● tim: the timer device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

3.2.3 sonata_timer_stop

Items	Description
Function	Stop a hardware timer
Parameters	● tim: the timer device
Return	N/A

3.2.4 sonata _timer_finalize

Items	Description
Function	De-initialize a TIMER interface, turn off a TIMER hardware interface
Parameters	● tim: the timer device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

3.2.5 sonata_timer_get

Items	Description
Function	Get hardware timer remain time
Parameters	● tim: the timer device
Return	Result: ● remain time ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

3.2.6 sonata_timer_reload

Items	Description
Function	Reload hardware timer value
Parameters	● tim: the timer device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

3.2.7 sonata_inter_clear

Items	Description
Function	Clear timer interrupt flag
Parameters	● TIMERX: the interface which should be cleared
Return	N/A

3.2.8 sonata_get_curvalue

Items	Description
Function	Get timer current value
Parameters	● tim: the timer device
Return	Result: timer current value

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

4. RTC

4.1 Functional description

There is only one RTC (real time clock) hardware in ASR560X, which is used to provide accurate time information. Structure **sonata_rtc_dev_t** is used to describe ASR560X RTC hardware, and the definition is shown below:

Table 4-1 sonata_rtc_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	RTC index, must be 0
config	sonata_rtc_config_t	timer configuration: format: time format DEC or BCD rtc_run_mode: ● RTC_RUN_TYPE_ALARM_1S ● RTC_RUN_TYPE_ALARM ● RTC_RUN_TYPE_1S
priv	void *	User-defined private data
sonata_rtc_time_struct	sonata_rtc_time_t	RTC data structure: year, month, day, hour, minute, second
rtc_handler_dummy	sonata_rtc_cb_handler_t	RTC handler callback function
arg	void *	Parameter of RTC handler callback function
Val_Alarm_Reload	uint32_t	Value of alarm

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

4.2 APIs

4.2.1 sonata_rtc_init

Items	Description
Function	Initialize the on board CPU real time clock
Parameters	● *sonata_rtc_init_struct: the RTC device
Return	N/A

4.2.2 sonata_rtc_get_time

Items	Description
Function	Get the value of time read from the on board real time clock
Parameters	● rtc: the RTC device ● time: pointer to a time structure
Return	Result: ● 0: success ● EIO: if an error occurs at any step

4.2.3 sonata_rtc_set_time

Items	Description
Function	Set RTC time to a new value
Parameters	● rtc: the RTC device ● time: pointer to a time structure
Return	Result: ● 0: success ● EIO: if an error occurs at any step

4.2.4 sonata_rtc_finalize

Items	Description
Function	De-initialize an RTC interface, turn off an RTC hardware interface
Parameters	● rtc: the RTC device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

5. ADC

5.1 Functional description

There are two kinds of ADC (analog to digital converter): AUX ADC and Audio ADC hardware in ASR560X. Audio ADC is used to sampling the audio data and AUX ADC is used to sampling other analog signal.

Structure **sonata_adc_dev_t** is used to describe ASR560X ADC hardware, and the definition is shown below:

Table 5-1 sonata_adc_dev_t structure

Member Variable Name	Type	Description
Mode	ADCMODESEL	The ADC mode ● SOFTFLOW (AUX ADC switching the channel by manual method) ● HARDFLOW (AUX ADC polling the channels by hardware) ● LIN_IN (AUD ADC) ● MIC_IN (AUD ADC)
SmpNums	uint16_t	The sampling times
priv AdcBuf	uint32_t *	The buf of saving the ADC data
Aux_channel	ADC_CH	The AUX ADC channel **ADC_CHNO_channel GPIO** ● AUXADC_CH0 PAD6 ● AUXADC_CH1 PAD7 ● AUXADC_CH2 PAD8 ● AUXADC_CH3 PAD9 ● AUXADC_CH4 PAD10 ● AUXADC_CH5 PAD11 ● AUXADC_CH6 PAD12 ● AUXADC_CH7 PAD13 ● ICTCH TMMT_CT ● IPTCH TMMT_PT ● VBATINCH VBAT
cb	sonata_adc_cb_handler_t	ADC handler callback function
arg	Void *	Parameter of ADC handler callback function
smp_rate	AUD_SMP_RATE	The AUD ADC sampling rate ● AUD_8k ● AUD_12k ● AUD_16k ● AUD_24k

ASR Confidential

Member Variable Name	Type	Description
		● AUD_32k ● AUD_48k ● AUD_64k ● AUD_96k ● AUD_192k ● AUD_11p025k ● AUD_22p05k ● AUD_44p1k ● AUD_88p1k ● AUD_176p4k
Micbia_ctl	State	AUD ADC mode config, enable or disable
gain	AUD_GAIN	AUD ADC signal gain ● G_0db ● G_3db ● G_6db ● G_9db ● G_12db ● G_15db ● G_18db ● G_21db ● G_24db ● G_27db ● G_30db ● G_33db ● G_36db ● G_39db ● G_42db ● G_45db
aux_poll_ptr	uint8_t *	Point to polling channel arry Only for HARDFLOW MODE
aux_poll_chans	uint8_t	Nums of AUX ADC polling channels Only for HARDFLOW MODE

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

5.2 APIs

5.2.1 sonata_adc_init

Items	Description
Function	Initialize the on board CPU real time clock
Parameters	● sonata_adc_init_struct: the ADC device
Return	N/A

5.2.2 sonata_aux_cmd

Items	Description
Function	Enable or disable the AUX ADC sampling
Parameters	● new_state: AUX ADC command
Return	N/A

5.2.3 sonata_aux_int

Items	Description
Function	Enable or disable the AUX ADC interrupts
Parameters	● int_mask: value of interrupt control register ● new_state: interrupt command
Return	N/A

5.2.4 sonata_get_aux_int_sta

Items	Description
Function	Get the AUX ADC interrupt status
Parameters	● void
Return	Result: the value of interrupt status register

5.2.5 sonata_clr_aux_int_sta

Items	Description
Function	Clear the AUX ADC interrupt status
Parameters	● aux_int: the value of interrupt clear register
Return	N/A

5.2.6 sonata_get_aux_fifo_sta

Items	Description
Function	Read the AUX ADC status
Parameters	● void
Return	Result: the value of AUX ADC FIFO status

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

5.2.7 sonata_aud_cmd

Items	Description
Function	Enable or disable the AUD ADC sampling
Parameters	● new_state: AUD ADC command
Return	N/A

5.2.8 sonata_aud_int

Items	Description
Function	Enable or disable the AUD ADC interrupts
Parameters	● int_mask: value of interupt control register ● new_state: interrupt command
Return	N/A

5.2.9 sonata_get_aud_int_sta

Items	Description
Function	Return the AUD ADC interrupt status
Parameters	● void
Return	Result: the value of interrupt status register

5.2.10 sonata_clr_aud_int_sta

Items	Description
Function	Clear the AUD ADC interrupt status
Parameters	● void
Return	N/A

5.2.11 sonata_get_aud_fifo_sta

Items	Description
Function	Get the AUD ADC FIFO status
Parameters	● void
Return	Result: the value of AUD ADC FIFO status

5.2.12 sonata_aux_dma_ctl

Items	Description
Function	Enable or disable the AUX ADC trigger DMA
Parameters	● new_state: AUX ADC trigger DMA command
Return	N/A

5.2.13 sonata_aud_dma_ctl

Items	Description
Function	Enable or disable the AUD ADC trigger DMA
Parameters	● new_state: AUD ADC trigger DMA command
Return	N/A

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

5.2.14 sonata_aud_init_step

Items	Description
Function	Initialize AUD ADC
Parameters	● sonata_adc_init_struct: the ADC device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

5.2.15 sonata_dma_adc_init

Items	Description
Function	Configure ADC to support DMA
Parameters	● sonata_adc_init_struct: the ADC device
Return	N/A

5.2.16 sonata_adc_init_SOFTFLOW

Items	Description
Function	Initialize AUX ADC SOFTFLOW mode
Parameters	● sonata_adc_init_struct: the ADC device
Return	N/A

5.2.17 sonata_adc_finalize

Items	Description
Function	De-initialize an ADC interface, turn off an ADC hardware interface
Parameters	● sonata_adc_init_struct: the ADC device
Return	N/A

5.2.18 sonata_get_aud_fifo_dat

Items	Description
Function	Get AUD ADC FIFO data
Parameters	● void
Return	Result: AUD ADC FIFO data

5.2.19 sonata_get_aux_fifo_dat

Items	Description
Function	Get AUX ADC FIFO data
Parameters	● void
Return	Result: AUX ADC FIFO data

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

6. PWM

6.1 Functional description

There are a total of 12 PWM (Pulse Width Modulation) channels in ASR560X, which can output square waveform with specific frequency and duty cycle. Structures **sonata_pwm_dev_t** and **sonata_cap_dev_t** are used to describe ASR560X PWM hardware, and the definitions are shown below:

Table 6-1 sonata_pwm_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	PWM channel index, between 0 to 11
config	sonata_pwm_config_t	pwm configuration: ● duty_cycle: the PWM duty cycle ● freq: the PWM frequency
priv	void *	User-defined private data

Table 6-2 sonata_cap_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	PWM CAP pin port ● PWM_CAP_CH0 PWM0 ● PWM_CAP_CH1 PWM2 ● PWM_CAP_CH2 PWM4 ● PWM_CAP_CH3 PWM6 ● PWM_CAP_CH4 PWM8 ● PWM_CAP_CH5 PWM10
config	sonata_cap_config_t	PWM CAP configuration ● cap_num: the cap num, max 65535 ● cpa_mode: the cap mode POSEDGE_EDGE_MODE NEGEDGE_EDGE_MODE BOTH_EDGE_MODE
priv	void *	User-defined private data

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

6.2 APIs

6.2.1 sonata_pwm_init

Items	Description
Function	Initialize a PWM pin
Parameters	● pwm: the PWM device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.2 sonata_pwm_start

Items	Description
Function	Start Pulse-Width Modulation signal output on a PWM pin
Parameters	● pwm: the PWM device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.3 sonata_pwm_stop

Items	Description
Function	Stop output on a PWM pin
Parameters	● pwm: the PWM device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.4 sonata_pwm_finalize

Items	Description
Function	De-initialize an PWM interface, turn off a PWM hardware interface
Parameters	● pwm: the PWM device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.5 sonata_cap_init

Items	Description
Function	Initialize a PWM CAP pin
Parameters	● cap: the PWM CAP device
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

6.2.6 sonata_cap_finalize

Items	Description
Function	De-initialize an PWM CAP interface, turn off an PWM CAP hardware interface
Parameters	● cap: the interface which should be de-initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.7 sonata_pwm_para_chg

Items	Description
Function	Change the para of pwm
Parameters	● pwm: the PWM device ● para: the para of pwm
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.8 sonata_pwmdiv_config

Items	Description
Function	Change PWM div cnt
Parameters	● div_cnt: PWM DIV CNT 0: DIV2 1: DIV4 2: DIV8 3: DIV16 4: DIV32 5: DIV64 6: DIV128 OTHERS: disable div ● enable: 0: disable div, OTHERS: enable div
Return	Result: ● 0: success ● EIO: if an error occurs at any step

6.2.9 sonata_pwm_clk_use_135M

Items	Description
Function	Select PWM clock source
Parameters	● void
Return	N/A

ASR Confidential

7. Flash

7.1 Functional description

ASR560X has one embedded SPI NOR flash with XIP function. Structure **sonata_logic_partition_t** is used to describe Flash memory mapping, and the definition is shown be low:

Table 7-1 sonata_logic_partition_t structure

Member Variable Name	Type	Description
partition_owner	sonata_flash_t	flash partition owner: ● FLASH_EMBEDDED ● FLASH_SPI ● FLASH_QSPI ● FLASH_MAX ● FLASH_NONE
partition_description	char *	flash partition description string
partition_start_addr	uint32_t	flash partition start address
partition_length	uint32_t	flash partition length in Bytes
partition_options	uint32_t	flash partition read and write permission

7.2 APIs

7.2.1 sonata_flash_init

Items	Description
Function	sonata flash init note: this function must be called before flash erase/write operation, also, this function must not be interrupted, it should be called with interrupt disabled
Parameters	NONE
Return	Result: ● 0: success ● EIO: if an error occurs at any step

7.2.2 sonata_flash_get_info

Items	Description
Function	Get the information of the specified flash area
Parameters	● in_partition: The target flash logical partition
Return	Result: sonata_logic_partition_t struct pointer

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

7.2.3 sonata_flash_find

Items	Description
Function	Find the target flash logical partition by name
Parameters	● name: Partition name
Return	Result: the target flash logical partition

7.2.4 sonata_flash_erase

Items	Description
Function	Erase an area on a flash logical partition
Parameters	● in_partition: The target flash logical partition which should be erased ● off_set: Start address of the erased flash area ● size: Size of the erased flash area
Return	Result: ● 0: success ● EIO: if an error occurs at any step

7.2.5 sonata_flash_write

Items	Description
Function	Write data to an area on a flash logical partition without erasing it
Parameters	● in_partition: The target flash logical partition which should be written ● off_set: Point to the start address that the data is written to, and point to the last unwritten address after this function is returned, so you can call this function several times without updating this start address. ● in_buf: point to the data buffer that will be written to flash ● in_buf_len: The length of the buffer
Return	Result: ● 0: success ● EIO: if an error occurs at any step

7.2.6 sonata_flash_erase_write

Items	Description
Function	Write data to an area on a flash logical partition with erasing it
Parameters	● in_partition: The target flash logical partition which should be written ● off_set: Point to the start address that the data is written to, and point to the last unwritten address after this function is returned, so you can call this function several times without updating this start address. ● in_buf: point to the data buffer that will be written to flash ● in_buf_len: The length of the buffer
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

7.2.7 sonata_flash_read

Items	Description
Function	Read data from an area on a flash to data buffer in RAM
Parameters	● in_partition: The target flash logical partition which should be read ● off_set: Point to the start address that the data is read, and point to the last unread address after this function is returned, so you can call this function several times without updating this start address. ● out_buf: Point to the data buffer that stores the data read from flash ● in_buf_len: The length of the buffer
Return	Result: ● 0: success ● EIO: if an error occurs at any step

7.3 Demo

Items	Description
Target	Show how to use the flash APIs and disable cache function for flash
Parameters	● CACHE_ENABLE: 0-disable cache function; 1-enable cache function ● TEST_PARTITION: The partition tested, can be changed to another one ● FLASH_TEST_OFFSET: The start address of tested area ● FLASH_TEST_SIZE: The size of tested area, can be changed to a larger size to check the overflow operation
Details	1. Initialize the flash 2. Get the information of the partition and print out 3. Write a buffer to the partition and read out again 4. Erase-Write a buffer to the partition and read out again 5. Erase the area, and read 100 times for the purpose of checking cache function
Notes	This demo should run in RAM.

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

8. Flash Encrypt

8.1 Functional description

The flash in ASR560X can be switched to encrypt mode. The flash switched to encrypt mode should be re-programmed before being used again. Be careful, this operation is **irreversible**.

8.2 APIs

8.2.1 sonata_flash_encrypt_status

Items	Description
Function	Get the encrypt status
Parameters	NONE
Return	Result: ● 0: un-encrypt ● 1: encrypt

8.2.2 sonata_flash_encrypt

Items	Description
Function	Switch flash to encrypt mode Note: This operation is **irreversible**
Parameters	NONE
Return	NONE

8.3 Demo

Items	Description
Target	Show how to switch the flash to encrypt mode
Parameters	● CASE_IMG_1: For generating a flash image which only prints a log ● CASE_ENCRYPT: Switch the flash to encrypt mode ● TEST_CASE: Which one of the two cases to run
Details	1. Set TEST_CASE to CASE_IMG_1, and program into flash. The log should be printed 2. Set TEST_CASE to CASE_ENCRYPT, and run in RAM. 3. Reset System to boot from flash, and it will fail to boot up. Because the un-encrypted software in flash can't be used in encrypt mode. 4. Set TEST_CASE to CASE_IMG_1 again, and program into flash. The system can boot up and print the log. At this time, the software in flash is encrypted.
Notes	This demo should run in RAM or Flash according to the Details. This operation is **irreversible.**

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

9. Flash Remappig

9.1 Functional description

You can exchange the address of two flash areas in ASR560X. It's very useful during OTA upgrade. This operation will be invalid after reboot.

9.2 APIs

9.2.1 sonata_flash_remapping

Items	Description
Function	Remap two areas of the flash
Parameters	● src_addr: The source flash area to remap ● dst_addr: The target flash area to remap ● size: Size of the flash area to remap
Return	NONE

9.3 Demo

Items	Description
Target	Show how to remap two areas of the flash
Parameters	● CASE_IMG_1: For generating a flash image which prints a signature log ● CASE_IMG_2: For generating another flash image which prints a signature log ● CASE_RUN_IMG1: Boot jumps to the address TEST_SRC_ADDR ● CASE_RUN_IMG1_MAPPING: Flash remaps and boot jumps to the address TEST_SRC_ADDR ● TEST_CASE: Which one of the four cases to run
Details	1. Set TEST_CASE to CASE_IMG_1, and program image 1 into TEST_SRC_ADDR 2. Set TEST_CASE to CASE_IMG_2, and program image 2 into TEST_DST_ADDR 3. Set TEST_CASE to CASE_RUN_IMG1, and run in RAM. Image 1 should be launched 4. Set TEST_CASE to CASE_RUN_IMG1_MAPPING, and run in RAM. Image 2 should be launched
Notes	This demo should run in RAM or Flash according to the Details. This operation will be invalid after reboot.

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

10. UART

10.1 Functional description

There are four identical UART devices in ASR560X, which support 5-8 data bits per character, optional parity bit and 1 or 2 stop bits. Structure **sonata_uart_dev_t** is used to describe ASR560X UART hardware, and the definition is shown below:

Table 10-1 sonata_uart_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	UART device index, must be 0, 1, 2, 3
config	sonata_uart_config_t	UART configuration: ● baud_rate: UART baud rate ● data_width: UART data bits per character DATA_5BIT DATA_6BIT DATA_7BIT DATA_8BIT ● parity: UART parity bits PARITY_NO PARITY_ODD PARITY_EVEN ● stop_bits: UART stop bits STOP_1BIT STOP_2BITS ● flow_control: UART flow control FLOW_CTRL_DISABLED FLOW_CTRL_CTS FLOW_CTRL_RTS FLOW_CTRL_CTS_RTS ● mode: UART mode TX_MODE RX_MODE TX_RX_MODE
priv	void *	User-defined private data

SR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

10.2 APIs

10.2.1 sonata_uart_init

Items	Description
Function	Initialize a UART interface
Parameters	● uart: the interface which should be initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.2 sonata_uart_set_tx_fifo_threshold

Items	Description
Function	Set UART TX FIFO threshold
Parameters	● UARTx: the interface which should be configured ● uart_fifo_level: FIFO level to be set
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.3 sonata_uart_set_rx_fifo_threshold

Items	Description
Function	Set UART RX FIFO threshold
Parameters	● UARTx: the interface which should be config ● uart_fifo_level: FIFO level to be set
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.4 sonata_uart_interrupt_config

Items	Description
Function	UART interrupt configuration
Parameters	● UARTx: the interface which should be configuration ● uart_int: interrupt bits ● new_state: interrupt state ENABLE DISABLE
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

10.2.5 sonata_uart_cmd

Items	Description
Function	Close or open UART function
Parameters	● UARTx: the interface which should be configured ● newstate: UART state ENABLE DISABLE
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.6 sonata_uart_send

Items	Description
Function	Transmit data on a UART interface
Parameters	● uart: the UART interface ● data: pointer to the start address of data ● size: number of bytes to transmit ● timeout: reserved to 0
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.7 sonata_uart_dma_config

Items	Description
Function	Configure UART to support DMA
Parameters	● uart: the UART interface ● dma_tx_rx_sel: the value of UART DMA config register ● new_state: DMA state ENABLE DISABLE
Return	Result: ● 0: success ● EIO: if an error occurs at any step

10.2.8 sonata_uart_start

Items	Description
Function	Enable UART and enable interrupt
Parameters	● UARTx: the interface which should be started
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

10.2.9 sonata_uart_stop

Items	Description
Function	Disable UART and disable interrupt
Parameters	● UARTx: the interface which should be stopped
Return	N/A

10.2.10 sonata_uart_finalize

Items	Description
Function	De-initializes a UART interface
Parameters	● UARTx: the interface which should be de-initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

11. I2C

11.1 Functional description

There are two identical I2C devices in ASR560X, which can be configured as master or slave. They also support two speed modes and 7-bit addressing mode. Structure **sonata_i2c_dev_t** is used to describe ASR560X I2C hardware, and the definition is shown below:

Table 11-1 sonata_i2c_dev_t structure

Member Variable Name	Type	Description
i2c_speed_mode	uint32_t	I2c speed mode ● I2C_MODE_STANDARD ● I2C_MODE_FAST
i2c_role	uint32_t	I2c master mode or slave mode ● I2C_MASTER ● I2C_SLAVE
i2c_mst_fifo_mode	uint32_t	i2c fifo mode enable ● I2c_mst_fifo_mode_EN ● I2c_mst_fifo_mode_DIS
i2c_slave_addr	uint32_t	I2c slave address
i2c_dma_function	uint32_t	I2c dma enable ● ENABLE ● DISABLE

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

11.2 APIs

11.2.1 sonata_i2c_unit_reset

Items	Description
Function	Reset I2C
Parameters	● I2Cx: the device for which the I2C port should be reset.
Return	Result: ● 0: success ● EIO: if an error occurs at any step

11.2.2 sonata_i2c_config_struct_init

Items	Description
Function	Initialize an I2C structure
Parameters	● pI2C_InitSturct: I2C configuration structure, default setting.
Return	N/A

11.2.3 sonata_i2c_init

Items	Description
Function	Initialize an I2C interface, prepare an I2C hardware interface for communication as a master or slave
Parameters	● I2Cx: the device for which the I2C port should be initialized ● pI2C_InitSturct: I2C configuration structure
Return	Result: ● 0: success ● EIO: if an error occurs at any step

11.2.4 sonata_i2c_master_write_data

Items	Description
Function	I2C master writes data to the specific device address
Parameters	● I2Cx: I2C device ● slave_addr: device address ● pwdata: data buffer to be sent ● wlen: data length
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

11.2.5 sonata_i2c_master_read_data

Items	Description
Function	I2C master reads data from the specific device address
Parameters	● I2Cx: I2C device ● slave_addr: device address ● data: buffer for storing data received ● size: data size
Return	Result: ● 0: success ● EIO: if an error occurs at any step

11.2.6 sonata_i2c_master_repeated_write_read

Items	Description
Function	I2C master repeats writing and reading data from the specific device address
Parameters	● I2Cx: I2C device ● slave_addr: device address ● pwdata: buffer of data to be sent ● rdata: buffer for storing the received data ● wlen: length of the data to be sent ● rlen: length of the received data
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

12. SPI

12.1 Functional description

There are two SPI devices in ASR560X. There are dedicated four lines (clk,cs,spi_tx,spi_rx) for each SPI controller, all of which support the Motorola SPI standard. Structure **sonata_spi_dev_t** is used to describe ASR560X SPI hardware, and the definition is shown below:

Table 12-1sonata_spi_dev_t structure

Member Variable Name	Type	Description
port	uint8_t	SPI index, must be 0,1
config	sonata_spi_config_t	SPI configuration: ● freq: SPI clk frequency ● mode: SPI mode SPI_ROLE_MASTER SPI_ROLE_SLAVE
priv	void *	User-defined private data

12.2 APIs

12.2.1 sonata_spi_init

Items	Description
Function	Initialize an SPI interface, set up the communication settings
Parameters	● spi: the device for which the SPI port should be initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

12.2.2 sonata_spi_send

Items	Description
Function	SPI device sends the specified sized data
Parameters	● spi: the device to send data ● data: buffer of data to be sent ● size: data size ● timeout: reserved to 0
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

12.2.3 sonata_spi_receive

Items	Description
Function	SPI device receives the specified sized data
Parameters	● spi: the device to receive data ● rx_data: buffer for receiving data ● size: data size ● timeout: reserved to 0
Return	Result: ● 0: success ● EIO: if an error occurs at any step

12.2.4 sonata_spi_interrupt_config

Items	Description
Function	SPI interrupt configuration
Parameters	● SPIx: the device for which the SPI port should be configured ● spi_interrupt: SPI interrupt bit ● new_state: interrupt state ENABLE DISABLE
Return	N/A

12.2.5 sonata_spi_cmd

Items	Description
Function	Open or close SPI function
Parameters	● spi: the device for which the SPI port shoud be opened or closed ● new_state: SPI state ENABLE DISABLE
Return	N/A

12.2.6 sonata_spi_finalize

Items	Description
Function	De-initialize an SPI interface
Parameters	● spi: the SPI device to be de-initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

12.2.7 sonata_spi_cpol_cpha_config

Items	Description
Function	Change SPI device work mode
Parameters	● spi: the device for which the SPI port's work mode should be changed ● mode: work mode, selectable from 0~3
Return	Result: ● 0: success ● EIO: if an error occurs at any step

12.2.8 sonata_spi_dma_config

Items	Description
Function	Configure SPI to support DMA
Parameters	● spi: the device for which the SPI port should be changed ● dma_tx_rx_sel: the value of SPI DMA config register ● new_state: DMA state ENABLE DISABLE
Return	Result: ● 0: success ● EIO: if an error occurs at any step

12.2.9 sonata_spi_datasize

Items	Description
Function	Configure SPI datasize
Parameters	● spi: the device for which the SPI port should be changed ● data_size: SPI data size
Return	Result: ● 0: success ● EIO: if an error occurs at any step

12.2.10 sonata_spi_trx_byte

Items	Description
Function	SPI device sends and receives one Byte data
Parameters	● spi: the device for which the SPI port should be changed ● tx_data: one Byte data ● timeout: reserved to 0
Return	Result: received data

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

13. eFuse

13.1 Functional description

The eFuse of ASR560X is 1k-bit one-time programmable memory, with the default value of 0. It can only be written from 0 to 1.

13.2 APIs

13.2.1 sonata_efuse_init

Items	Description
Function	eFuse init, must be called before read/write operation. If eFuse write is needed, ldo25_open must be set to 1. If only eFuse read is needed, then ldo25_open should be set to 0.
Parameters	● ldo25_open: 1- open ldo25 for eFuse write operation
Return	N/A

13.2.2 sonata_efuse_byte_read

Items	Description
Function	Read one eFuse Byte
Parameters	● addr: eFuse addr for read, from 0x00 to 0x7F
Return	Result: Data in corresponding eFuse address

13.2.3 sonata_efuse_word_read

Items	Description
Function	Read one eFuse word
Parameters	● addr: eFuse addr for read, from 0x00 to 0x7C
Return	Result: Data in corresponding eFuse address

13.2.4 sonata_efuse_byte_write

Items	Description
Function	Program one eFuse Byte
Parameters	● addr: eFuse addr for write, from 0x00 to 0x7F ● data: data to be written
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

13.2.5 sonata_efuse_word_write

Items	Description
Function	Program one eFuse word
Parameters	● addr: eFuse addr for write, from 0x00 to 0x7C ● data: data to be written
Return	N/A

13.2.6 sonata_efuse_multi_read

Items	Description
Function	Read multiple eFuse bytes
Parameters	● start_addr: eFuse addr for write, from 0x00 to 0x7F ● size_in_bytes: how many Bytes to be read ● *pData: where eFuse data is stored
Return	N/A

13.3 Demo

Items	Description
Target	Show how to write or read eFuse data
Parameters	● EFUSE_TEST_WRITE: 1 for write. BE CAREFUL, eFuse bit cann't be rewritten after writing 1 ● EFUSE_READ_COUNT: The number of eFuse in Bytes for testing
Details	1. Write some Bytes into eFuse area 2. Read one Byte from an eFuse address 3. Read one word from an eFuse address 4. Read some Bytes from eFuse area
Notes	This demo can run in RAM or Flash.

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

14. AES

14.1 Functional description

The AES API can encrypt the user data and decrypt it back with a key.

14.2 APIs

14.2.1 sonata_aes_encrypt

Items	Description
Function	AES encrypts the input data with a key.
Parameters	● key: The key of AES. Only the first 16 Bytes used. If less than 16, '0' will be filled in. ● key_size: The size of key ● input: The input data to be encrypted ● input_size: The size of input ● output: The output data encrypted ● output_size: The address of output size, which should be large enough to hold the output. A number larger than 'sizeof(aes_input) + 16' is suggested. When encryption succeedds, output_size will hold the actual size of output
Return	Result: When encryption succeeds, the encrypted data is read into output, the size of which is output_size.

14.2.2 sonata_aes_decrypt

Items	Description
Function	AES decrypts the input data with a key.
Parameters	● key: The key of AES. Only the first 16 Bytes are used. If less than 16, '0' will be filled in. ● key_size: The size of key ● input: The input data to be decrypted ● input_size: The size of input, which must be multiple of 16 ● output: The output data decrypted ● output_size: The address of output size, which should be large enough to hold the output. A number not less than 'sizeof(aes_input) + 16' is suggested. When decryption succeeds, output_size will hold the actual size of output.
Return	Result: When decryption succeeds, the decrypted data is read into output, the size of which is output_size.

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

14.3 Demo

Items	Description
Target	Show how to encrypt or decrypt user data with the AES APIs.
Parameters	● key: The key of AES. Only the first 16 Bytes are used. If less than 16, '0' will be filled in. ● aes_input: The input data for testing ● aes_output: The output data from AES encryption or decryption
Details	1. Encrypt aes_input with aes_key, and the result put in aes_output 2. Encrypt aes_output with aes_key, and the result put in aes_output, which is multiple of 16. The first input_size of aes_output should be equal to the original aes_input.
Notes	This demo can run in RAM or Flash.

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

15. SRAM Mapping

15.1 Functional description

ASR560X provides four SRAM partition schemes for BLE and MCU SRAM requirements with the default scheme of 96K+32K. If there's more SRAM requirement for SoC, another 3 partition schemes can be used: 104K+24K, 112K+16K and 120K+8K.

15.2 Demo

Items	Description
Target	Show how to switch the SRAM partition schemes
Parameters	● _estack: Set _estack in cfg/gcc.ld according to your requirement, and _estack can't be greater than 0x2001E000.
Details	1. The partition of RAM will be configured during initialization. The logic is shown as follows, and the max value in each section is suggested ● _estack SOC BLE ● <=0x20018000 96K 32K ● (0x20018000, 0x2001A000] 96K+8K 24K ● (0x2001A000, 0x2001C000] 96K+16K 16K ● (0x2001C000, 0x2001E000] 96K+24K 8K 2. Run the demo with a specific _estack, an address of a stack variable will be printed, and the whole BLE address space will be gone through.
Notes	This demo can run in Flash, however, the initial code must run first before RAM initialization.

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

16. I2S

16.1 Functional description

Structure **sonata_i2s_dev_t** is used to describe ASR560X I2S hardware, and the definition is shown below:

Table 16-1 sonata_i2s_dev_t structure

Member Variable Name	Type	Description
I2s_sample_rate	uint32_t	I2S sample rate
i2s_mclk_src	uint32_t	clock source of mclk
i2s_mclk_div	uint32_t	clock division of mclk
i2s_ws	uint32_t	i2s bit width
i2s_role	uint8_t	I2S role ● I2S_ROLE_MASTER ● I2S_ROLE_SALVE
i2s_word_size	uint8_t	I2S word size
i2s_tx_en	uint8_t	TX enable ● ENABLE ● DISABLE
i2s_rx_en	uint8_t	RX enable ● ENABLE ● DISABLE
i2s_fifo_threshold	uint8_t	the threshold of TX/RX FIFO
i2s_mode	uint8_t	I2S mode, only Philips mode supported

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

16.2 APIs

16.2.1 sonata_i2s_init

Items	Description
Function	Initialize an I2S interface, set up the communication settings
Parameters	● I2Sx: the device for which the I2S port should be initialized ● pI2S_struct: sonata_i2s_dev_t structure
Return	NONE

16.2.2 sonata_i2s_struct_init

Items	Description
Function	Initialize a sonata_i2s_dev_t structure used by sonata_i2s_init
Parameters	● pI2S_struct: sonata_i2s_dev_t structure
Return	NONE

16.2.3 sonata_i2s_send_data

Items	Description
Function	I2S device sends the specified sized data
Parameters	● I2Sx: the device to send data ● left_chan_data: left channel data ● right_chan_data: right channel data ● size: data size
Return	NONE

16.2.4 sonata_i2s_interrupt_config

Items	Description
Function	I2S interrupt configuration
Parameters	● I2Sx: the device to configure ● i2s_interrupt: I2S interrupt bit I2S_INTERRUPT_TXFO I2S_INTERRUPT_TXFE I2S_INTERRUPT_RXFO I2S_INTERRUPT_RXDA ● new_state: interrupt state ENABLE DISABLE
Return	NONE

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

16.2.5 sonata_i2s_interrupt_clear

Items	Description
Function	Clear I2S interrupt configuration
Parameters	● I2Sx: the device to configure ● i2s_interrupt: I2S interrupt bit I2S_INTERRUPT_TXFO I2S_INTERRUPT_RXFO
Return	NONE

16.2.6 sonata_i2s_cmd

Items	Description
Function	Open or close I2S function
Parameters	● I2Sx: I2S device to be opened or closed ● new_state: I2S state ENABLE DISABLE
Return	NONE

16.2.7 sonata_i2s_tx_block_cmd

Items	Description
Function	Open or close I2S TX block
Parameters	● I2Sx: I2S device ● new_state: I2S state ENABLE DISABLE
Return	NONE

16.2.8 sonata_i2s_rx_block_cmd

Items	Description
Function	Open or close I2S RX block
Parameters	● I2Sx: I2S device ● new_state: I2S state ENABLE DISABLE
Return	NONE

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

16.2.9 sonata_i2s_tx_channel_cmd

Items	Description
Function	Open or close I2S TX channel
Parameters	● I2Sx: I2S device ● new_state: I2S state ENABLE DISABLE
Return	NONE

16.2.10 sonata_i2s_rx_channel_cmd

Items	Description
Function	Open or close I2S RX channel
Parameters	● I2Sx: I2S device ● new_state: I2S state ENABLE DISABLE
Return	NONE

16.2.11 sonata_i2s_master_clock_cmd

Items	Description
Function	Open or close I2S master clock
Parameters	● I2Sx: I2S device ● new_state: I2S state ENABLE DISABLE
Return	NONE

16.2.12 sonata_i2s_receive_data

Items	Description
Function	Get data from one of I2S channels
Parameters	● I2Sx: I2S device ● lr: I2S channel 0: left channel 1: right channel
Return	uint32_t the data got from one of I2S channels

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

17. Keypad

17.1 Functional description

The ASR560X has a 14*8 keypad interface, Structure **sonata_keypad_dev_t** is used to describe ASR560X keypad hardware, and the definition is shown below:

Table 17-1 sonata_keypad_dev_t structure

Member Variable Name	Type	Description
config	sonata_keypad_dev_t	keypad configuration: ● t_debounce: set debounce time ● row_size: row size ● col_size:col size ● src_clk: source clock
priv	void *	User-defined private data

17.2 APIs

17.2.1 sonata_keypad_init

Items	Description
Function	Initialize an keypad interface, set up the communication settings
Parameters	● KP_init_struct: keypad init struct
Return	Result: ● 0: success ● EIO: if an error occurs at any step

17.2.2 sonata_keypad_get_single_key_row

Items	Description
Function	keypad device gets row number when single key is passed
Parameters	● Void
Return	Result: row number

17.2.3 sonata_keypad_get_single_key_col

Items	Description
Function	keypad device gets col number when single key is passed
Parameters	● Void
Return	Result: col number

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.

17.2.4 sonata_keypad_get_multi_key_stat

Items	Description
Function	keypad device gets col&row state when multi keys are passed
Parameters	● KeyStat: col&row state
Return	N/A

17.2.5 sonata_keypad_interrupt_config

Items	Description
Function	keypad interrupt configuration
Parameters	● new_state: interrupt state ENABLE DISABLE
Return	N/A

17.2.6 sonata_keypad_cmd

Items	Description
Function	Open or close keypad function
Parameters	● new_state: keypad state ENABLE DISABLE
Return	N/A

17.2.7 sonata_keypad_get_status

Items	Description
Function	Get keypad pressed status
Parameters	● void
Return	Result: keypad pressed status

17.2.8 sonata_keypad_wkup

Items	Description
Function	Wakeup keypad
Parameters	● KP_init_struct: keypad init struct
Return	N/A

17.2.9 sonata_keypad_get_multi_key_stat

Items	Description
Function	Get multi keypad status
Parameters	● KeyStat : pointer to keystat array
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

17.2.10 sonata_keypad_scan

Items	Description
Function	Scan keypad
Parameters	● void
Return	N/A

17.2.11 sonata_keypad_wkup_scan

Items	Description
Function	Wakeup keypad and scan keypad
Parameters	● void
Return	N/A

ASR Confidential

18. LPUART

18.1 Functional description

The ASR560X has one LPUART port, which supports 5-8 data bits per character, optional parity bits and 1or 2 stop bits. Structure **sonata_lpuart_dev_t** is used to describe ASR560X LPUART hardware, and the definition is shown below:

Table 18-1 sonata_lpuart_dev_t structure

Member Variable Name	Type	Description
lpuart_wakeup_pin	uint8_t	LPUART index
baudrate	uint32_t	LPUART baud rate
data_width	lpuart_data_width_t	LPUART data bits per character ● DATA_5BIT ● DATA_6BIT ● DATA_7BIT ● DATA_8BIT
parity	lpuart_parity_t	LPUART parity bit ● PARITY_NO ● PARITY_ODD ● PARITY_EVEN
stop_bits	lpuart_stop_bits_t	LPUART stop bits ● STOP_1BIT ● STOP_2BITS
low_level_wakeup	bool	RX pin wakeup level ● ENABLE ● DISABLE
start_wakeup	bool	RX start bit wakeup ● ENABLE ● DISABLE
rx_done_wakeup	bool	RX receive data done wakeup ● ENABLE ● DISABLE

R Confidential

Copyright © ASR Microelectronics Co., Ltd.

18.2 APIs

18.2.1 sonata_lpuart_init

Items	Description
Function	Initialize a LPUART interface
Parameters	● lpuart: the interface which should be initialized ● uart_init: LPUART configuration structure
Return	Result: ● 0: success ● EIO: if an error occurs at any step

18.2.2 sonata_lpuart_config_rx

Items	Description
Function	Enable or disable LPUART RX function
Parameters	● lpuart: the LPUART interface ● new_state: LPUART TX state ENABLE DISABLE
Return	N/A

18.2.3 sonata_lpuart_config_tx

Items	Description
Function	Enable or disable LPUART TX function
Parameters	● lpuart: the LPUART interface ● new_statel: LPUART TX state ENABLE DISABLE
Return	N/A

18.2.4 sonata_lpuart_config_interrupt

Items	Description
Function	LPUART interrupt configuration
Parameters	● lpuart: the LPUART interface ● lpuart_int: interrupt bits ● new_state: interrupt state ENABLE DISABLE
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

18.2.5 sonata_lpuart_receive_data

Items	Description
Function	LPUART device receives data
Parameters	● lpuart: the LPUART interface
Return	Result: received data

18.2.6 sonata_lpuart_send_data

Items	Description
Function	LPUART device sends data
Parameters	● lpuart: the LPUART interface ● data: data should be sent
Return	N/A

18.2.7 sonata_lpuart_get_sr0_state

Items	Description
Function	Get LPUART device SR0 register status
Parameters	● lpuart: the LPUART interface ● sr0_state: value of LPUART SR0 LPUART_SR0_START_VALID_STATE LPUART_SR0_RX_DONE_STATE LPUART_SR0_START_INVALID_STATE LPUART_SR0_PARITY_ERR_STATE LPUART_SR0_STOP_ERR_STATE LPUART_SR0_RX_OVERFLOW_STATE
Return	Result: ● True: the status of LPUART SR0 is true ● False: the status of LPUART SR0 is false

18.2.8 sonata_lpuart_clear_sr0_state

Items	Description
Function	Clear LPUART device SR0 register status
Parameters	● lpuart: the LPUART interface ● sr0_state: value of LPUART SR0 LPUART_SR0_START_VALID_STATE LPUART_SR0_RX_DONE_STATE LPUART_SR0_START_INVALID_STATE LPUART_SR0_PARITY_ERR_STATE LPUART_SR0_STOP_ERR_STATE LPUART_SR0_RX_OVERFLOW_STATE
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

18.2.9 sonata_lpuart_get_sr1_state

Items	Description
Function	Get LPUART device SR1 register status
Parameters	● lpuart: the LPUART interface ● sr1_state: value of LPUART SR1 LPUART_SR1_WRITE_SR0_STATE LPUART_SR1_WRITE_CR0_STATE LPUART_SR1_RX_NOT_EMPTY_STATE LPUART_SR1_TX_EMPTY_STATE LPUART_SR1_TX_DONE_STATE
Return	Result: ● True: the status of LPUART SR1 is true ● False: the status of LPUART SR1 is false

18.2.10 sonata_lpuart_clear_sr1_state

Items	Description
Function	Clear LPUART device SR1 register status
Parameters	● lpuart: the LPUART interface ● sr1_state: value of LPUART SR1 LPUART_SR1_WRITE_SR0_STATE LPUART_SR1_WRITE_CR0_STATE LPUART_SR1_RX_NOT_EMPTY_STATE LPUART_SR1_TX_EMPTY_STATE LPUART_SR1_TX_DONE_STATE
Return	N/A

18.2.11 sonata_lpuart_rxpin_sel

Items	Description
Function	Select LPUART RX pin
Parameters	● uart_init: the LPUART interface
Return	N/A

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

19. Quadrature Decoder

19.1 Functional description

The ASR560X has three Quadrature decoder interfaces. Structure **sonata_quaddec_dev_t** is used to describe ASR560X QUADDEC hardware, and the definition is shown below:

Table 19-1 sonata_quaddec_dev_t structure

Member Variable Name	Type	Description
port	sonata_quaddec_port_t	Quaddec port: QUADDEC_X, QUADDEC_Y, QUADDEC_Z
config	sonata_quaddec_config_t	UART configuration: ● qd_cnt_max: set qd max count ● qd_cnt_ori: set qd origin count ● div: clock division QD_CLK_DIV_1 QD_CLK_DIV_2 QD_CLK_DIV_4 QD_CLK_DIV_8 QD_CLK_DIV_16 QD_CLK_DIV_32 QD_CLK_DIV_64 QD_CLK_DIV_128 ● res: resolution QD_CNT_RES_X1 QD_CNT_RES_X2 QD_CNT_RES_X4
priv	void *	User-defined private data

19.2 APIs

19.2.1 sonata_quaddec_init

Items	Description
Function	Initialize a quaddec interface
Parameters	● quaddec: the interface which should be initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

19.2.2 sonata_quaddec_get_dir

Items	Description
Function	Get quaddec counting direction
Parameters	● quaddec: the quaddec interface ● dir: counting direction
Return	Result: ● 0: success ● EIO: if an error occurs at any step

19.2.3 sonata_quaddec_get_cnt

Items	Description
Function	Get the current counter of quaddec device
Parameters	● quaddec: the quaddec interface ● cnt: counting value
Return	Result: ● 0: success ● EIO: if an error occurs at any step

19.2.4 sonata_quaddec_finalize

Items	Description
Function	De-initialize a quaddec interface
Parameters	● quaddec: the interface which should be de-initialized
Return	Result: ● 0: success ● EIO: if an error occurs at any step

19.2.5 sonata_quaddec_set_seq

Items	Description
Function	Set sequence of x, y , z quaddec input channel
Parameters	● quaddec: the quaddec interface ● seq: hex: 0x00543210 -- default value. bit 0~3: quaddec_x_a input channel, it can be 0~5; bit 4~7: quaddec_x_b input channel, it can be 0~5; bit 8~11: quaddec_y_a input channel, it can be 0~5; bit 12~15: quaddec_y_b input channel, it can be 0~5; bit 16~19: quaddec_z_a input channel, it can be 0~5; bit 20~23: quaddec_z_b input channel, it can be 0~5; bit 24~31: reserved.
Return	Result: ● 0: success ● EIO: if an error occurs at any step

ASR Confidential

 Copyright © ASR Microelectronics Co., Ltd.

19.2.6 sonata_quaddec_get_seq

Items	Description
Function	Return sequence of x, y, z quaddec input channel
Parameters	● quaddec: the interface to get sequence from
Return	Result: ● sequence of x, y, z quaddec input channel ● EIO: if an error occurs at any step

ASR Confidential

Copyright © ASR Microelectronics Co., Ltd.